

ESIGELEC

Development of a Cloud-Based Video Fingerprint Collection System for an Insurance Company

Cloud Computing

Student: Enzo Westphal Tacla

Discipline: Cloud Computing

Date: 28/09/2026

Contents

1	Introduction	3
1.1	Project Context	3
1.2	Objectives	3
1.3	Proposed Solution	4
2	System Architecture	5
2.1	Overall Architecture	5
2.2	Technology Choices	5
3	Encoder	5
3.1	Camera Frame Acquisition	6
3.2	Frame Processing and Recording Representation	6
3.3	SHA-256 Fingerprint Generation	7
3.4	Dynamic Cloud Transmission	7
3.5	Handling Network Interruptions	8
3.6	Automatic Data Retention	8
3.7	Encoder Technical Choices	9
4	Decoder	9
4.1	Recording Session Retrieval	10
4.2	Integrity Verification Algorithm	10
4.3	Reliability of the Integrity Mechanism	11
4.4	Decoder Results	11
4.5	Decoder Design Decisions	12
5	Cloud Infrastructure	13
5.1	Cloud Backend	13
5.2	PostgreSQL Database	14
5.3	Cloud Data Model	14
5.4	Row Level Secuirty	15
5.5	Automatic Retention Edge Function	15
5.6	Scheduled Cleanup	15
5.7	Cloud Infrastructure Design Decisions	16
6	Results and Demonstration	16
6.1	Normal Operation	16
6.2	Modified Fingerprint Detection	17
6.3	Missing Fingerprint Detection	18
6.4	Network Interruption and Recovery	19
6.5	Automatic Retention Test	19
6.6	Overall Results	20

1 Introduction

1.1 Project Context

Dashcams continuously capture information about what happens during a vehicle journey, making them a useful source of evidence when investigating road incidents or handling insurance claims. However, continuously transmitting and storing complete video recordings or individual image frames in the cloud can require significant storage capacity and network bandwidth.

This project explores an alternative approach based on video fingerprints. The system consists of two main applications: an Encoder/Transmitter, which runs on the driver's smartphone, and a Decoder, which allows an insurer to retrieve and verify the recorded information.

The Encoder captures frames from the camera stream at regular intervals. Each frame is temporarily converted into a JPEG and processed locally to generate a SHA-256 fingerprint. The JPEG itself is not permanently stored or uploaded to the cloud. Instead, only its fingerprint and the metadata needed to identify its position within the recording session are transmitted.

To provide additional integrity protection, the fingerprints are connected through a hash chain. This creates a relationship between consecutive fingerprints and their positions in the recording. The Decoder can later retrieve the sequence and check whether the stored fingerprints remain consistent with the integrity information originally generated by the Encoder.

1.2 Objectives

The main objective of this project is to develop a cloud-based solution for generating, transmitting, storing, and verifying fingerprints derived from dashcam recordings without permanently storing the original video frames in the cloud.

The Encoder captures frames directly from the smartphone camera and generates a SHA-256 fingerprint for each one. Every fingerprint is associated with a recording session, sequence number, timestamp, and Driver ID before being transmitted to the cloud.

The system is also designed to handle temporary network interruptions. If communication with the cloud is unavailable, the fingerprint information is stored locally in a persistent IndexedDB queue. Once the connection is restored, the Encoder automatically sends the pending fingerprints.

On the insurer side, the Decoder retrieves the fingerprints associated with a Driver ID and recording session and checks the integrity of the sequence. It can identify fingerprints that remain valid, fingerprints whose integrity information has been modified, and gaps caused by missing sequence numbers.

Finally, fingerprints are automatically removed from the cloud after a configurable retention period. For demonstration purposes, this period is set to 10 minutes.

1.3 Proposed Solution

The Encoder and Decoder were developed as React and TypeScript web applications using Vite, with Supabase providing the cloud backend. PostgreSQL is used to store the fingerprints and their associated metadata.

On the Encoder side, the browser's MediaDevices API provides access to the camera. Frames are periodically extracted from the video stream using an HTML canvas and temporarily converted into JPEG data. The binary content of each frame is then processed using the Web Crypto API to generate a SHA-256 fingerprint:

$$H_n = \text{SHA256}(\text{Frame}_n)$$

where (H_n) is the fingerprint generated from frame (n).

The fingerprints are then linked using a hash chain. For the first fingerprint:

$$C_1 = \text{SHA256}(H_1)$$

For each subsequent fingerprint:

$$C_n = \text{SHA256}(C_{n-1} \parallel H_n)$$

where (C_n) is the chain hash for the current fingerprint and (C_{n-1}) is the chain hash from the previous fingerprint.

Only the fingerprint, chain hash, sequence number, timestamp, Driver ID, and session ID are sent to PostgreSQL. Once the fingerprint has been generated, the temporary JPEG data is discarded and is not stored in Supabase.

If the cloud connection is unavailable, the fingerprint information is stored locally in IndexedDB using Dexie. When connectivity returns, the Encoder automatically processes the pending queue and sends the stored information to the cloud.

On the Decoder side, the insurer retrieves the fingerprint sequence for a recording session. The Decoder uses the stored fingerprints to recalculate the expected hash chain and compares the results with the chain values generated by the Encoder. A matching value is classified as VALID, while a mismatch is classified as MODIFIED. Gaps in the expected sequence_number are identified as MISSING.

The Decoder also provides an overall result for the recording session. A session is considered VERIFIED when all expected fingerprints pass the integrity check. If a fingerprint is modified or a sequence number is missing, the session is classified as COMPROMISED.

2 System Architecture

2.1 Overall Architecture

The Dashcam system follows a client-cloud architecture with three main components: the Encoder/Transmitter, the Supabase cloud backend, and the Decoder.

The Encoder runs on the driver’s smartphone and captures frames from the camera at regular intervals. It generates a SHA-256 fingerprint for each frame, builds the hash chain, and sends the resulting information to the cloud.

Supabase connects the Encoder and Decoder and provides persistent storage for the generated data. In the final architecture, the captured frames are not stored in the cloud. Only the fingerprints, chain hashes, and related session metadata are stored in the PostgreSQL database.

On the insurer side, the Decoder retrieves this information from Supabase and verifies the integrity of the fingerprint sequence by independently checking the hash chain.

2.2 Technology Choices

The Encoder and Decoder were developed as mobile-compatible web applications using React, TypeScript, and Vite. This approach allows the system to run directly in a web browser while still providing access to features such as the device camera and cryptographic functions, without the need to develop a separate native mobile application.

The main technologies used in the system are:

Component	Technology
Encoder	React, TypeScript, Vite, MediaDevices API, Web Crypto API, IndexedDB
Cloud Infrastructure	Supabase, PostgreSQL
Decoder	React, TypeScript, Vite, Web Crypto API

Table 1: Development environment

The following sections describe each component individually, including its implementation, algorithms, data management strategy, and role in the complete system.

3 Encoder

The Encoder/Transmitter is designed to run on the driver’s smartphone while the device is mounted on the vehicle windshield. It captures frames from the camera, generates cryptographic fingerprints, builds the hash chain, and sends the resulting information to the cloud. It is also designed to handle temporary network interruptions without losing pending fingerprint data.

3.1 Camera Frame Acquisition

Before starting a recording, the user enters a Driver ID, which is associated with all fingerprints generated during that session. The same identifier is later used by the Decoder to retrieve the corresponding recordings.

The camera is activated using the Start Camera button. The Encoder accesses the device camera through the browser's MediaDevices API and displays the live stream on the screen. When the user selects Start Recording, the application generates a new UUID for the session and initializes a sequence counter to preserve the original order of the generated fingerprints.

Rather than creating a complete video file, the Encoder periodically captures individual frames from the camera stream. Each frame is drawn onto an HTML canvas and temporarily converted into a JPEG 'Blob' with a quality setting of '0.9'.

This JPEG representation is used only to generate the fingerprint. Once the fingerprint has been calculated, the temporary frame data is discarded and is never uploaded to the cloud.

3.2 Frame Processing and Recording Representation

A recording session is represented as an ordered sequence of cryptographic fingerprints, rather than as a stored video file or a collection of JPEG images. Each fingerprint record contains:

- `driver_id`;
- `session_id`;
- `sequence_number`;
- `timestamp`;
- `hash`;
- `chain_hash`.

All fingerprints from the same recording share a `session_id`, while the `sequence_number` identifies their position in the sequence and preserves the original order in which the frames were captured.

The original frame is needed only during local processing. Once its SHA-256 fingerprint and chain hash have been generated, the temporary JPEG data is discarded.

As a result, only the fingerprints and their associated metadata need to be transmitted and stored in the cloud. This reduces both the amount of data sent over the network and the storage required by the system.

3.3 SHA-256 Fingerprint Generation

For each captured frame, the Encoder generates a SHA-256 fingerprint using the browser's Web Crypto API. The temporary JPEG Blob is first converted into an ArrayBuffer, which contains the binary data used to calculate the hash: *crypto.subtle.digest('SHA-256', frameBuffer)*

The resulting value represents the fingerprint of that specific frame. Since the hash is calculated directly from the JPEG binary data, any change to the frame content would produce a different fingerprint.

The resulting 256-bit hash is converted into a hexadecimal string. For a captured frame (F_n), its fingerprint can be represented as:

$$H_n = \text{SHA256}(F_n)$$

In addition to the frame fingerprint, the Encoder generates a second cryptographic value called the chain hash. This value connects each fingerprint to the previous one within the same recording session.

For the first fingerprint:

$$C_1 = \text{SHA256}(H_1)$$

For each subsequent fingerprint:

$$C_n = \text{SHA256}(C_{n-1} \parallel H_n)$$

where:

- H_n is the SHA-256 fingerprint of the current frame;
- C_{n-1} is the chain hash from the previous fingerprint;
- C_n is the chain hash generated for the current fingerprint;
- $\parallel$ represents concatenation.

The hash therefore represents the fingerprint generated directly from the captured frame, while the chain_hash links that fingerprint to the previous one in the sequence. The Decoder later uses these values to check whether the stored fingerprint sequence remains consistent with the chain originally generated by the Encoder.

3.4 Dynamic Cloud Transmission

Fingerprint records are sent to the cloud while the recording is still in progress, so the Encoder does not need to wait for the session to finish.

Once the frame fingerprint and chain hash have been generated, the Encoder creates a record containing the Driver ID, session ID, sequence number, timestamp, frame hash, and

chain hash. This information is then inserted into the Supabase PostgreSQL fingerprints table.

The original JPEG frame is not transmitted. Only the cryptographic values and the metadata needed to identify and verify each fingerprint are sent to the cloud. This also reduces the amount of pending data that needs to remain on the driver’s device during normal operation.

3.5 Handling Network Interruptions

A smartphone used as a dashcam may temporarily lose its internet connection. To avoid losing fingerprints during these interruptions, the Encoder uses a persistent offline queue implemented with IndexedDB, with Dexie providing the local database interface.

When the connection to Supabase is available, fingerprint records are sent directly to the cloud database. If transmission fails, the fingerprint and its associated metadata are stored locally in the ‘pendingFingerprints’ queue.

Since the original JPEG frames are not transmitted or stored, the offline queue only needs to keep the fingerprint information required for a later upload. This keeps the amount of locally stored data relatively small.

When the browser detects that the connection has been restored, the Encoder automatically processes the pending queue and attempts to send each record to Supabase. A fingerprint is removed from IndexedDB only after it has been successfully transmitted. If another attempt fails, the record remains in the queue and can be retried later.

This behavior was tested by disconnecting the network during an active recording session. Fingerprints generated while the connection was unavailable were stored in IndexedDB. Once connectivity was restored, the pending records were automatically sent to Supabase and removed from the local queue.

3.6 Automatic Data Retention

The system includes a retention mechanism to prevent fingerprint information from remaining indefinitely on the device or in the cloud. For demonstration purposes, the retention period is set to 10 minutes.

On the Encoder side, expired records are automatically removed from the offline queue based on their timestamps. This prevents fingerprints that are no longer within the retention period from being transmitted after a long network interruption.

Data already stored in Supabase is handled separately by the cloud backend. A scheduled cleanup process periodically checks the database and removes fingerprints that are older than the configured limit. This process is described in more detail in the Cloud Infrastructure section.

By handling retention both locally and in the cloud, expired fingerprint data can be removed whether it is still waiting to be transmitted or has already been stored in the backend.

3.7 Encoder Technical Choices

Several design decisions were made to meet the project requirements. The first was to represent a recording using cryptographic fingerprints instead of the captured frames themselves. Each frame is needed only temporarily to generate its SHA-256 hash. Once this fingerprint has been calculated, the JPEG data can be discarded. This reduces both the amount of data transmitted by the smartphone and the storage required in the cloud.

Individual frame hashes, however, do not provide information about the relationship between fingerprints in a recording. For this reason, the Encoder also generates a hash chain, where each `chain_hash` depends on the current frame fingerprint and the chain hash of the previous record.

This gives the Decoder a value that can be independently recalculated for each fingerprint. Using the previous chain hash and the current frame hash, the Decoder can calculate the expected value and compare it with the `chain_hash` generated by the Encoder.

The `sequence_number` has a different but complementary role. While the hash chain checks the consistency of the stored fingerprints, sequence numbers make missing records visible. For example, a sequence containing ‘1, 2, 4, 5’ shows that fingerprint ‘3’ is missing. Together, these mechanisms allow the Decoder to distinguish between missing and modified data.

During verification, the Decoder uses the stored chain hash of the preceding record when checking each existing fingerprint. This makes it possible to identify where an inconsistency occurs without automatically marking all subsequent fingerprints as modified. For example, if fingerprint 3 has been altered, it can be classified as ‘MODIFIED’, while fingerprint 4 can still be checked using the original chain information stored for fingerprint 3. The session as a whole is still classified as ‘COMPROMISED’.

Persistent local storage was also chosen to handle temporary network interruptions. IndexedDB, accessed through Dexie, keeps pending fingerprints available until they can be transmitted. Since only fingerprints and metadata are stored in the queue, there is no need to persist the temporary JPEG frames.

Overall, the recording representation combines three elements with different roles: the frame hash represents the captured data, the chain hash links consecutive fingerprints, and the sequence number preserves their expected order and reveals missing records. Together, they provide the information needed for integrity verification without requiring the original frames to be stored.

4 Decoder

The Decoder is the application used by the insurer to retrieve the fingerprint information generated by the Encoder and verify the integrity of the stored recording sequence.

Since the original camera frames are not stored in the cloud, the Decoder performs the verification using the fingerprints, hash-chain information, and sequence numbers

associated with each recording session.

The Decoder was developed as a React and TypeScript web application and retrieves the required fingerprint data directly from the Supabase PostgreSQL database.

4.1 Recording Session Retrieval

The Decoder uses the Driver ID to find the recording sessions associated with a specific driver. The user enters the same identifier that was provided to the Encoder before the recording started.

The application searches the fingerprints table and groups the returned records by session_id. Each available session is displayed with its starting timestamp and the number of fingerprints it contains.

When a session is opened, the Decoder retrieves its fingerprint records and orders them by sequence_number. Each record contains the information needed for verification, including:

- sequence number;
- timestamp;
- frame fingerprint (hash);
- stored chain hash (chain_hash).

4.2 Integrity Verification Algorithm

The main role of the Decoder is to independently recalculate the integrity proof associated with each stored fingerprint.

For the first fingerprint, the Encoder generates:

$$C_1 = \text{SHA256}(H_1)$$

The Decoder performs the same calculation using the retrieved frame fingerprint:

$$C'_1 = \text{SHA256}(H_1)$$

and compares the result with the stored chain_hash:

$$C'_1 \stackrel{?}{=} C_1$$

For each subsequent fingerprint, the Decoder uses the stored chain hash of the previous record together with the current frame fingerprint:

$$C'_n = \text{SHA256}(C_{n-1} \parallel H_n)$$

The calculated value C'_n is then compared with the chain_hash C_n originally stored by the Encoder:

$$C'_n \stackrel{?}{=} C_n$$

If the values match, the fingerprint is classified as VALID. If they differ, it is classified as MODIFIED.

Sequence continuity is checked separately using the sequence_number. The Decoder keeps track of the expected number and detects any gaps in the retrieved sequence. For example, if fingerprint 2 is followed by fingerprint 4, fingerprint 3 is identified as MISSING.

This approach allows the Decoder to distinguish between a fingerprint that is present but fails the integrity check and a fingerprint that is completely missing from the recording sequence.

4.3 Reliability of the Integrity Mechanism

The verification mechanism uses SHA-256 to generate consistent cryptographic values from the information produced during capture. If a frame fingerprint is changed, the hash-chain calculation will produce a different result with extremely high probability.

The Decoder does not simply trust the chain_hash stored in the database. Instead, it independently calculates the expected value using the current frame fingerprint and the stored chain hash of the previous record. The result is then compared with the value originally generated by the Encoder.

Sequence numbers provide an additional check. Because they define the expected order of the fingerprints, the Decoder can identify missing records even when there is no fingerprint available to verify.

However, the current implementation provides integrity consistency checking rather than authentication of the Encoder. If someone with sufficient database privileges replaced both the fingerprints and their corresponding chain hashes with a completely consistent new sequence, the Decoder would not have an independent trusted value to detect the change.

A production version could strengthen this protection by adding cryptographic authentication, such as digital signatures or a keyed message authentication code (MAC), with the authentication credentials kept separate from the fingerprint data stored in the cloud.

Within the scope of this project, the implemented mechanism is intended to detect missing fingerprints and inconsistencies between the stored fingerprints and their corresponding hash-chain values.

4.4 Decoder Results

The Decoder was tested under the main integrity scenarios covered by the project.

In the first test, an unchanged session was retrieved from Supabase. The Decoder recalculated the chain hashes and obtained the same values stored by the Encoder. All fingerprints were classified as VALID, and the session was reported as VERIFIED.

Modification detection was tested by manually changing the hash of an intermediate fingerprint while keeping its original chain_hash. When the session was retrieved again, the Decoder recalculated the expected chain value and detected the mismatch. The affected fingerprint was classified as MODIFIED, and the session was reported as COMPROMISED.

Missing-data detection was tested by deleting an intermediate fingerprint record. The Decoder identified the gap in the sequence_number and added a MISSING result for the absent position.

Both conditions were also tested at the same time. With fingerprint 2 removed and fingerprint 3 modified, the Decoder produced the following result:

Fingerprint 1: VALID
Fingerprint 2: MISSING
Fingerprint 3: MODIFIED

These tests show that the Decoder can distinguish between valid, modified, and missing fingerprints while preserving their expected order. The session-level result also provides a clear indication of whether the complete fingerprint sequence passed the integrity check.

4.5 Decoder Design Decisions

A central design decision was to make the Decoder work entirely with fingerprints and integrity metadata, without requiring access to the original video frames. This follows the Encoder architecture, where frames are used only temporarily to generate fingerprints and are then discarded.

The Decoder therefore does not compare stored images with their original versions. Instead, it checks whether the fingerprint sequence stored in the cloud remains consistent with the integrity information generated by the Encoder.

Another important decision was to keep sequence verification and cryptographic verification separate. A fingerprint that no longer exists cannot be checked through a hash calculation. For this reason, the sequence_number is used to detect MISSING records, while the hash chain is used to verify fingerprints that are present.

The verification algorithm also uses the stored chain hash of the previous record when calculating the expected value for the current fingerprint. This helps identify the specific record where an inconsistency occurs.

For example, if fingerprint 3 is modified and the Decoder uses the newly calculated invalid chain value to verify fingerprint 4, the error would propagate through the remaining sequence:

Fingerprint 1: VALID
Fingerprint 2: VALID
Fingerprint 3: MODIFIED

Fingerprint 4: MODIFIED

Fingerprint 5: MODIFIED

Instead, the Decoder uses the original stored `chain_hash` from fingerprint 3 when verifying fingerprint 4. This allows the inconsistency to remain localized:

Fingerprint 1: VALID

Fingerprint 2: VALID

Fingerprint 3: MODIFIED

Fingerprint 4: VALID

Fingerprint 5: VALID

Session integrity: COMPROMISED

This approach makes the verification results easier to interpret because the Decoder identifies the fingerprint where the inconsistency is detected rather than automatically propagating the same status to every subsequent record. The session is still classified as COMPROMISED, since at least one fingerprint failed the integrity check.

5 Cloud Infrastructure

The cloud infrastructure provides the communication and persistence layer between the Encoder and Decoder. The project uses Supabase as its backend platform, with PostgreSQL storing the fingerprints and metadata generated by the Encoder and later retrieved by the Decoder.

The captured JPEG frames are not stored in the cloud. They are used temporarily by the Encoder to generate the fingerprints and are discarded immediately afterward. As a result, the cloud only needs to store the relatively small fingerprint records and their associated metadata.

The backend is also responsible for managing data retention independently of the Encoder and Decoder. This allows expired fingerprint records to be automatically removed even when neither application is running.

5.1 Cloud Backend

Supabase was selected as the cloud backend because it provides the main services required by the project within a single platform.

The implementation uses:

- **Supabase PostgreSQL** to store fingerprints and their associated metadata;
- **Row Level Security (RLS)** to control frontend access to the database;
- **Supabase Edge Functions** to manage server-side data retention;
- **Supabase Cron** to run the cleanup process at scheduled intervals.

Both applications use the same backend but have different roles. The Encoder inserts new fingerprint records as they are generated, while the Decoder retrieves the stored information to identify recording sessions and verify their integrity.

The Encoder and Decoder therefore do not need to communicate directly. Supabase acts as the shared backend between fingerprint generation and later verification.

5.2 PostgreSQL Database

Frame metadata is stored in the fingerprints table. The implemented schema is:

Column	Type	Purpose
id	bigint	Automatically generated record identifier
session_id	uuid	Identifies the recording session
sequence_number	integer	Defines the frame position within the session
timestamp	timestampz	Time associated with fingerprint generation
hash	text	SHA-256 fingerprint generated by the Encoder
created_at	timestampz	Time associated with fingerprint generation
driver_id	text	Driver identifier
chain_hash	text	Integrity proof linking the fingerprint with the previous chain state

Table 2: Frames metadata

Each recording session contains multiple fingerprint records that share the same `session_id`. The `sequence_number` defines the order in which the fingerprints were generated and also allows the Decoder to detect gaps when a record is missing.

The `hash` and `chain_hash` have different roles. The `hash` is the SHA-256 fingerprint generated from the captured frame, while the `chain_hash` links that fingerprint to the previous state of the chain and is later used during integrity verification.

5.3 Cloud Data Model

The cloud stores a lightweight representation of each recording rather than the recording itself. For example, if five frames are captured during a session, five fingerprint records are stored in the database. All of these records share the same `session_id` and Driver ID, while their `sequence_number` identifies their position in the recording.

This approach separates the recording content from its integrity representation. The cloud does not store the original images or enough information to reconstruct the video. Instead, it keeps only the fingerprints and metadata needed by the Decoder to verify the consistency of the recorded sequence.

5.4 Row Level Security

Row Level Security (RLS) is enabled for the fingerprints table to control access from the Encoder and Decoder. The Encoder is allowed to insert new fingerprint records, while the Decoder can retrieve the information needed for integrity verification.

Both applications use the Supabase publishable key, which can safely be included in the frontend when access is controlled by the configured RLS policies. Privileged credentials are not included in the client-side application.

Operations that require higher permissions, such as automatically deleting expired fingerprint records, are handled by the server-side Edge Function. The required credentials remain in the Supabase environment and are never exposed to the browser.

This keeps administrative access on the backend while giving the Encoder and Decoder only the permissions required for their respective operations.

5.5 Automatic Retention Edge Function

Data retention is performed by a Supabase Edge Function named: `cleanup-expired-frames`.

During each execution, the cleanup function calculates the expiration threshold based on the current time and searches the fingerprints table for records with a capture timestamp older than that limit. Only records with a valid `frame_path` are included in the Storage cleanup.

For expired frames, the function collects their Storage paths and deletes the corresponding JPEG files from the `dashcamframes` bucket. The database records are removed only after the Storage deletion has completed successfully.

Because this cleanup requires administrative permissions, it runs with server-side credentials rather than the access provided to the Encoder and Decoder. The `service_role` was therefore granted the database privileges needed to perform these operations.

5.6 Scheduled Cleanup

Automatic invocation of the retention function is managed through Supabase Cron. The scheduled job is named: `cleanup-expired-dashcam-frames`.

The cleanup process runs once every minute. This interval defines how often the system checks for expired data, while the retention period itself is set to 10 minutes. As a result, a frame becomes eligible for deletion once its capture timestamp is more than 10 minutes old and is normally removed during the next scheduled cleanup.

The 10-minute retention period was chosen for demonstration purposes so that the complete deletion process could be observed within a short time. A different retention period can be configured directly in the Edge Function without requiring changes to the Encoder or Decoder.

5.7 Cloud Infrastructure Design Decisions

One of the main architectural changes made during development was the decision to stop storing captured frames permanently in the cloud. The initial implementation uploaded JPEG frames to Supabase Storage and kept their paths in PostgreSQL. This allowed the Decoder to download the images and recalculate their SHA-256 hashes, but it also meant that the backend had to store every captured frame.

In the final architecture, frames are used only temporarily by the Encoder to generate their fingerprints. Once this process is complete, the JPEG data is discarded and only the cryptographic information and related metadata are sent to the cloud. This reduces both network traffic and cloud storage requirements while keeping the recording content separate from its integrity representation.

PostgreSQL was retained because the remaining data is small and structured. Driver IDs, session IDs, sequence numbers, timestamps, fingerprints, and chain hashes fit naturally into a relational database and can be efficiently retrieved by the Decoder.

Another important decision was to manage data retention server-side. If cleanup depended on the Encoder or Decoder being active, expired records could remain in the database when neither application was running. A scheduled Edge Function avoids this dependency and allows the backend to manage retention independently.

Retention is also handled at two stages of the data lifecycle. The Encoder removes expired fingerprints that are still waiting in its local offline queue, while the cloud backend removes records that have already been successfully transmitted. This prevents outdated fingerprint data from remaining indefinitely in either location.

Finally, the cloud backend acts as the intermediary between the Encoder and Decoder rather than requiring direct communication between them. This allows recording and verification to happen at different times and on different devices, while both applications work with the same stored session information.

6 Results and Demonstration

The complete system was tested from fingerprint generation in the Encoder to integrity verification in the Decoder. The tests covered normal operation, temporary network interruptions, automatic data retention, and the detection of modified or missing fingerprints.

6.1 Normal Operation

The first test evaluated the complete system under normal network conditions. A Driver ID was entered in the Encoder, the camera was activated, and a new recording session was started.

As frames were captured, the Encoder generated a SHA-256 fingerprint and chain hash for each one and sent the resulting records to Supabase while the recording was still in progress. The database confirmed that all records from the same session shared the same `session_id` and had increasing `sequence_number` values.

The same Driver ID was then used in the Decoder to retrieve the session and verify the fingerprint sequence. Since no data had been modified or removed, all calculated chain values matched the values stored by the Encoder.

The Decoder therefore reported:

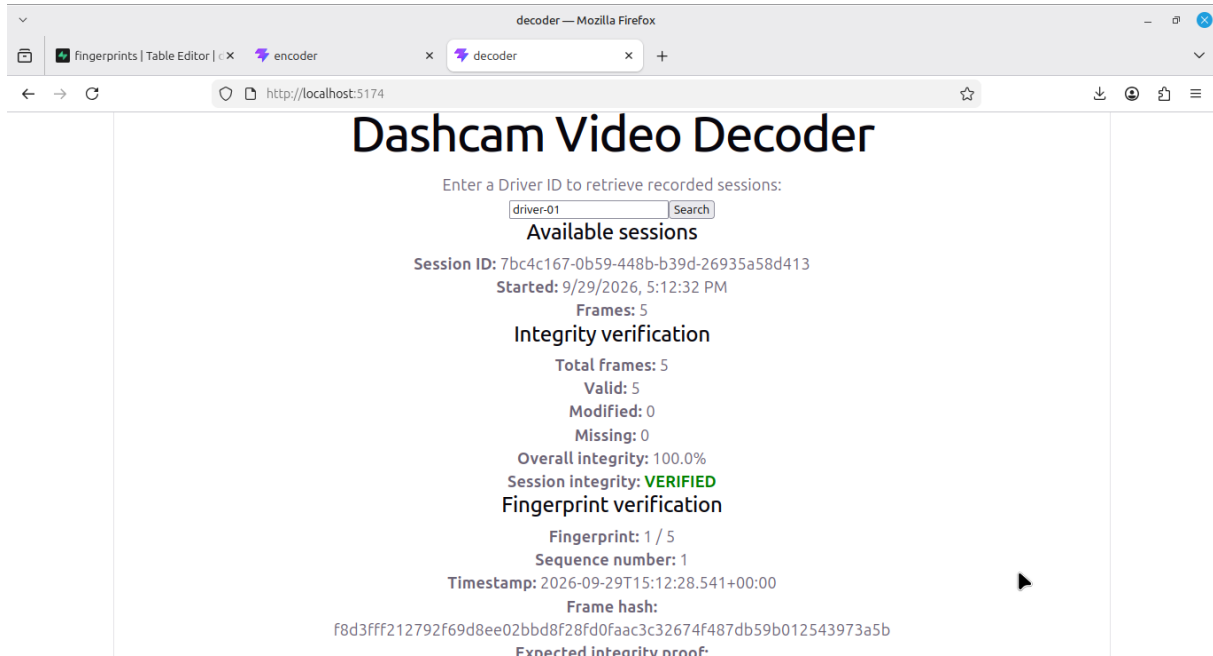


Figure 1: Valid Test

This test confirmed the complete normal workflow between the Encoder, cloud back-end, and Decoder.

6.2 Modified Fingerprint Detection

Modification detection was tested by deliberately changing the hash of an intermediate fingerprint in PostgreSQL while keeping its original chain_hash.

When the session was retrieved again, the Decoder used the modified fingerprint to recalculate the expected chain value. Since the result no longer matched the chain_hash originally generated by the Encoder, the fingerprint was classified as:

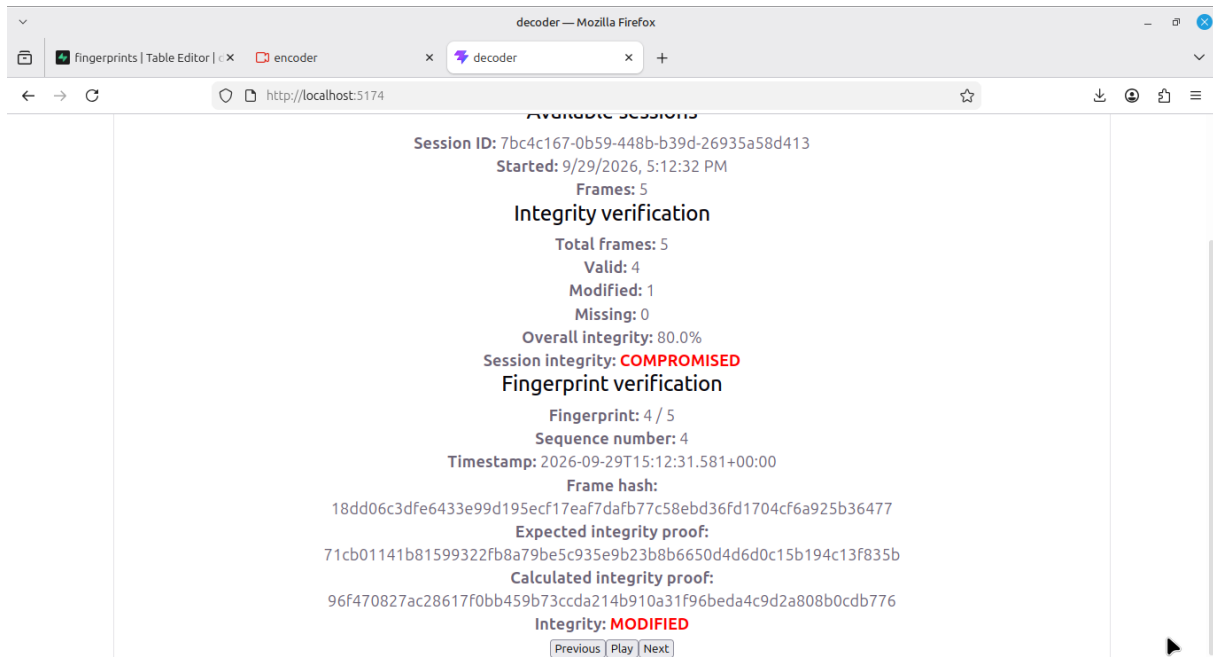


Figure 2: Valid Test

The remaining fingerprints were still checked individually, allowing the Decoder to identify the specific record where the inconsistency occurred.

6.3 Missing Fingerprint Detection

The missing-data mechanism was tested by deliberately deleting an intermediate fingerprint from a recording session. For example, after removing fingerprint 2, the database contained a sequence equivalent to: 1, 3, 4, 5.

When the session was retrieved, the Decoder detected the gap between sequence numbers identified as:

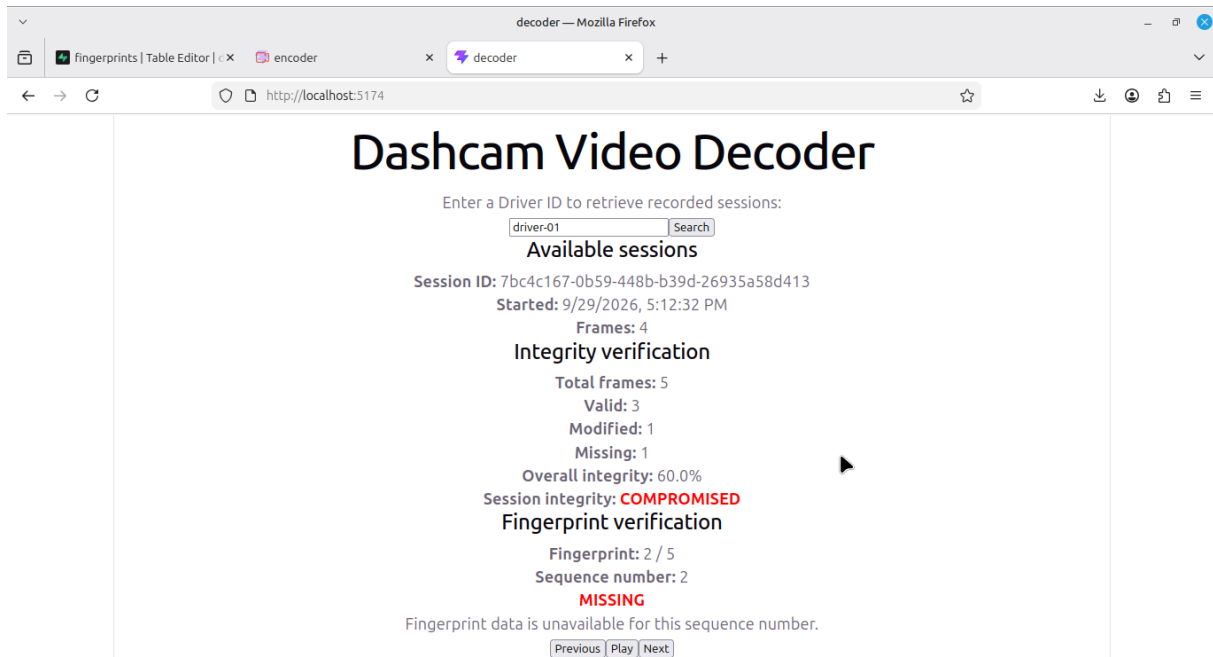


Figure 3: Valid Test

6.4 Network Interruption and Recovery

The offline mechanism was tested by disconnecting the network while the Encoder was actively generating fingerprints. Since the connection to Supabase was unavailable, the generated records were stored in the persistent IndexedDB pendingFingerprints queue instead of being discarded.

The test confirmed that the pending fingerprints remained available locally throughout the interruption. Once the connection was restored, the Encoder automatically processed the queue and transmitted the records to Supabase. After each successful transmission, the corresponding item was removed from IndexedDB.

The uploaded records preserved their original session information and sequence numbers, confirming that temporary network interruptions can be handled without losing the fingerprint data generated during that period.

6.5 Automatic Retention Test

The automatic retention mechanism was tested using the 10-minute retention period configured for the project. Fingerprint records were generated by the Encoder and confirmed to be present in PostgreSQL, while the scheduled cleanup process continued to run once per minute.

After the fingerprints became more than 10 minutes old, the server-side cleanup automatically removed the expired records from the database.

This test confirmed that data retention is handled independently by the cloud backend and continues to operate even when the Encoder and Decoder are not running.

6.6 Overall Results

The implemented prototype successfully demonstrated the main objectives of the project. The Encoder captured camera frames, generated SHA-256 fingerprints, built the hash chain, transmitted fingerprint information to the cloud, and preserved pending records during temporary network interruptions. The cloud backend stored the fingerprint-based representation of each recording. It also automatically removed expired records according to the configured retention period.

On the insurer side, the Decoder successfully retrieved recording sessions and identified VALID, MODIFIED, and MISSING fingerprints. It also provided an overall session result of either VERIFIED or COMPROMISED.

Together, the tests demonstrate the complete lifecycle implemented by the system: frames are captured and processed locally, fingerprints are generated and transmitted to the cloud, pending data is preserved during connection failures, stored sessions can later be verified by the Decoder, and expired information is automatically removed.

7 Conclusion

This project implemented a cloud-based video fingerprint collection and integrity verification system composed of an Encoder, a cloud backend, and a Decoder.

Instead of permanently storing the original video or individual JPEG frames, the system represents each recording as an ordered sequence of cryptographic fingerprints. The Encoder temporarily captures frames from the camera, generates a SHA-256 fingerprint for each one, and connects consecutive fingerprints through a hash chain. Only these cryptographic values and their associated metadata are transmitted to the cloud.

The implementation also demonstrated that the system can handle temporary network interruptions. Fingerprints that cannot be transmitted immediately are stored locally in IndexedDB and automatically sent to the cloud when connectivity returns. Once stored in the backend, they remain available for verification until the configured retention period expires, after which they are automatically removed.

On the insurer side, the Decoder retrieves recording sessions and independently checks the consistency of their fingerprint sequences. It can identify ‘VALID’, ‘MODIFIED’, and ‘MISSING’ fingerprints and provide an overall session result of ‘VERIFIED’ or ‘COMPROMISED’. The tests showed that combining SHA-256 fingerprints, hash chaining, and sequence numbering provides a practical way to detect inconsistencies and missing records without storing the original captured frames in the cloud.

The current prototype does have limitations. The hash chain provides integrity consistency checking, but it does not authenticate the origin of the fingerprints if an attacker is able to modify both the stored fingerprints and their corresponding chain hashes. A production version could address this limitation by introducing digital signatures or a keyed authentication mechanism, together with stronger user authentication and authorization.

Overall, the project demonstrates an alternative approach to conventional cloud dash-

cam storage. Rather than retaining the recording itself, the cloud stores a lightweight cryptographic representation that can later be checked for integrity. The prototype successfully covers the complete process from camera capture and fingerprint generation to cloud transmission, offline recovery, integrity verification, and automatic data retention.

References

- [1] Mozilla Developer Network (MDN). *MediaDevices: getUserMedia() method*. Available at: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>. Accessed September 2026.
- [2] Mozilla Developer Network (MDN). *Web Crypto API*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API. Accessed September 2026.
- [3] Dexie.js. *Dexie.js Documentation*. Available at: <https://dexie.org/docs/Dexie.js>. Accessed September 2026.
- [4] Supabase. *Edge Functions*. Available at: <https://supabase.com/docs/guides/functions>. Accessed September 2026.
- [5] Supabase. *Cron: Schedule Recurring Jobs*. Available at: <https://supabase.com/docs/guides/cron>. Accessed September 2026.